PRINCETON UNIVERSITY PRESS

Chapter Title: FOREST FIRES

Book Title: Natural Complexity

Book Subtitle: A Modeling Handbook

Book Author(s): Paul Charbonneau

Published by: Princeton University Press

Stable URL: https://www.jstor.org/stable/j.ctt1vwmgzt.9

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at https://about.jstor.org/terms

Princeton University Press is collaborating with JSTOR to digitize, preserve and extend access to *Natural Complexity*

JSTOR

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

6

FOREST FIRES

Chapter 4 introduced some of the remarkable properties of randomly produced percolation clusters. These clusters were entirely static, "frozen" objects, their structure determined once and for all by the specific realization of random deviates used to fill the lattice.

Can any "natural" process generate dynamically something conceptually resembling a percolation cluster? The answer is yes, as exemplified by the forest-fire model investigated in this chapter. Its ecological inspiration is probably as far removed as it could be from flow through porous media or phase transitions, yet at a deeper level it does represent an instance of dynamical percolation.

6.1 Model Definition

The forest-fire model is, fundamentally, a probabilistic CA. Sticking again to a 2-D Cartesian lattice, each node (i, j) is assigned a state $s_{i,j}$ which can take one of three possible values: "empty," "inactive," and "active." Starting from an empty lattice ($s_{i,j} = 0$ for all i, j), the nodal variable evolves in discrete time

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

steps ($s_{i,j}^{n} \to s_{i,j}^{n+1}$) according to a set of local rules, some of a stochastic nature:

Rule 1. An empty node can become occupied with probability p_{g} (stochastic).
Rule 2. An inactive node can be activated with probability p_{f} (stochastic).
Rule 3. An inactive node becomes active if one or more of its nearest neighbors was active at the preceding iteration (deterministic).
Rule 4. Active nodes become empty at the following iteration (deterministic).

The ecological inspiration of the model should be obvious: inactive nodes represent trees; active nodes are burning trees; Rule 1 is tree growth; Rule 2 is a tree being ignited by lightning; Rule 3 is fire jumping from one tree to a neighboring tree; and Rule 4 is destruction of a tree by fire. You have probably anticipated already that successive ignition of trees by a burning neighbor can lead to the propagation of a burning "front" across the lattice, i.e., an "avalanche" of burning trees. This expectation is certainly borne out, but as with the simple sandpile model considered in the preceding chapter, the spatiotemporal evolution of the system holds quite a few surprises in store for us.

6.2 Numerical Implementation

The Python source code listed in figure 6.1 is a minimal implementation of the forest-fire model, again in the sense that it sacrifices coding conciseness and execution speed to conceptual clarity and readability. The overall structure is similar to the sandpile code of figure 5.2, but the simulation is now performed on a 2-D Cartesian lattice of size `N`×`N`. Burning (active) nodes are assigned the numerical value `2`, while occupied (inactive) nodes are set to `1`, and empty nodes to `0`. The temporal iteration is governed by the outer fixed-length loop starting on line 15, inside

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

```
1  # FOREST-FIRE MODEL ON 2D CARTESIAN LATTICE
2  import numpy as np
3  import matplotlib.pyplot as plt
4  #-----------------------------------------------------------------------
5  N     =100                                    # Lattice size
6  p_g   =1.e-3                                  # Growth probability
7  p_f   =1.e-5                                  # Lightning probability
8  n_iter=25000                                  # Number of temporal iterations
9  #-----------------------------------------------------------------------
10 dx=np.array([-1,0,1,1,1,0,-1,-1])             # Template arrays
11 dy=np.array([-1,-1,-1,0,1,1,1,0])
12 grid=np.zeros([N+2,N+2],dtype='int')          # Initialize lattice: no trees
13 trees=0                                       # Tree counter
14
15 for iterate in range(0,n_iter):                # temporal iteration
16     update=np.zeros([N+2,N+2],dtype='int')     # evolution array
17     burn=0                                     # burning tree counter
18     # scan lattice to flag which trees must grow, ignite or vanish
19     for i in range(1,N+1):
20         for j in range(1,N+1):
21             if grid[i,j] == 1:                 # there is a tree on this node
22                 if 2 in grid[i+dx[:],j+dy[:]]: # 1 or more burning neighbor
23                     update[i,j]=1              # ignite
24                     burn+=1
25                 if np.random.uniform() < p_f:  # lightning strikes (maybe)
26                     update[i,j]=1              # ignite
27                     burn+=1
28             if grid[i,j] == 2:                 # remove trees already burning
29                 update[i,j]=-2
30                 trees-=1
31             if grid[i,j] == 0:                 # empty node
32                 if np.random.uniform() < p_g:  # grow tree (maybe)
33                     update[i,j]=1
34                     trees+=1
35    # end of lattice scan
36
37     grid+=update                               # synchronous update lattice
38     print("iteration {0}, trees {1}, burn {2}.".format(iterate,trees,burn))
39 # end of temporal loop
40 plt.imshow(grid,interpolation="nearest")       # display final state
41 plt.show()
42 # END
```

Figure 6.1. A minimal implementation of the forest-fire model in the Python programming language.

which really all the coding action is sitting. Take note of the following:

1. The simulation begins with an empty lattice: all nodal values in `grid` are set to 0 (line 12), using Python/NumPy's array creation-and-initialization function `zeros`.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

2. As with the DLA code of chapter 3, the 2-D arrays `grid` and `update` are padded with an outer frame of ghost nodes which always remain empty, but allow nodes at the real edges of the lattice to be tested for ignition in the same manner as interior nodes. Consequently, even though the lattice array is of size `(N+2)`×`(N+2)`, loops over the lattice run from `1` to `N`, meaning in Python `range(1,N+1)` on lines 19–20, as per the loop range and array element numbering conventions in the Python programming language. See appendix D.1 if needed.
3. Ghost nodes retain the value 0 throughout the simulation; you may think of this as equivalent to the simulation domain being enclosed within four scrupulously well-maintained fire trenches.
4. Once again the nodes that are to grow a tree, catch fire, or become empty are first identified in the first block of `for` loops, and the needed changes (`+1` for tree growth, line 33; `+1` for igniting an existing tree, either by lightning (line 23) or via a burning neighbor (line 26); and `-2` for a burned tree vanishing (line 29)) are stored in the 2-D array `update`. This work array is reset to zero at the beginning of each temporal iteration (line 16).
5. The lattice update is later carried out synchronously, at the end of the temporal iteration loop (line 37).
6. The relative coordinates of the 8 nearest neighbors to any node are stored in the template arrays `dx` and `dy` (lines 10–11). See appendix D.1 if needed.
7. Again note, on line 22, the Python-specific instruction

   ```
   if 2 in grid[i+dx[:],j+dy[:]]:
   ```

   and its built-in implicit loop, to check whether there is at least one burning tree in the set of nearest neighbors

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

to node (i, j), as defined by the template arrays `dx` and `dy`.

8. Note that lightning can still strike while a fire is burning; this model is operating in "running" rather than "stop-and-go" mode.

In case you have not noticed it already, this forest-fire model is at the core of the algorithm introduced in chapter 4 for the tagging of clusters on the percolation lattice (if you are not convinced, compare figures 4.3 and 6.1). Occupied nodes are the trees; tree growth is turned off, and random ignition by lightning strikes is replaced by systematic ignition of as-yet-untagged occupied nodes. The ensemble of trees burned by each such ignition is a cluster, and the largest fire maps are the largest cluster on the lattice.

Getting back to the forest-fire model per se, clearly the rules governing the lattice evolution are quite simple, and only Rule 3 actually involves nearest-neighbor contact. Moreover, the model involves only two free parameters, namely, the tree growth probability p_g and the lightning probability p_f. Nonetheless, as these two parameters are varied, the model can generate a surprisingly wide range of behaviors, hard to anticipate on the basis of its defining dynamical rules.

6.3 A Representative Simulation

Figure 6.2 shows the triggering, growth, and decay of a large fire in a representative forest-fire model simulation on a small 100×100 lattice, with parameter values $p_g = 10^{-3}$ and $p_f = 10^{-5}$. This simulation had already been running for many thousands of iterations, and so had reached a statistically stationary state.[1] The

[1] In this forest-fire model this is best ascertained by tracking the total number of trees on the lattice, until it levels off to a stable mean value. Note also that whatever the initial conditions, the duration of the initial transient phase increases rapidly with decreasing p_g and p_f.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

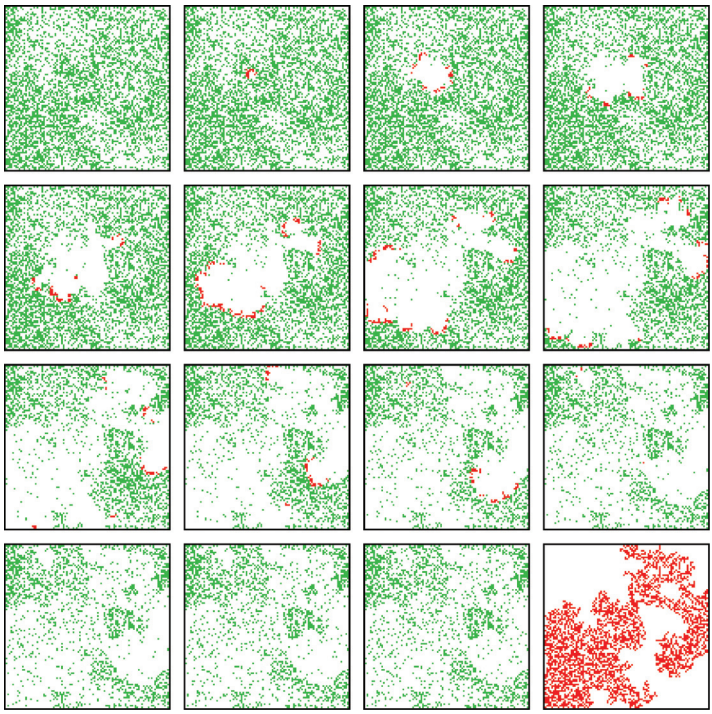

Figure 6.2. A sequence of snapshots, taken 10 iterations apart, of a 100×100 lattice in a simulation of the forest-fire model running with $p_g = 10^{-3}$ and $p_f = 10^{-5}$. Empty nodes are left white, nodes occupied but inactive are green, and active nodes are red. Here lightning has struck a bit left and up of the lattice center, two iterations prior to the second snapshot. The resulting burning front subsequently sweeps through a large fraction of the lattice. The bottom-right frame shows the location of all trees having burned in this fire. Notice also the small fire, triggered by a second lightning strike, ignited in the upper-left portion of the lattice a few iterations prior to the eleventh frame (third column in third row from top).

figure shows a sequence of snapshots taken 10 iterations apart, going left to right from top to bottom. A few iterations prior to the second snapshot, lightning has struck a bit up and left from

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

the lattice center. Fire activity propagates from node to node, at an average speed determined by the density of trees but never exceeding one lattice spacing (horizontally and/or vertically) per iteration, as per Rule 3. The combustion front is initially almost circular, but later evolves into a far more convoluted shape as the fire sweeps across the lattice, reflecting the substantial spatial variations of tree density in the pre-fire lattice configuration. This heterogeneity is itself a consequence of previous fires having burned across the lattice in the more or less distant past (see the last snapshot in figure 6.2). Even though fires are triggered by a stochastic process (Rule 2 above), past fire activity influences the evolution of current fires.

Figure 6.2 illustrates well the disparity of timescales characterizing the forest-fire model. The shortest is the "dynamical" timescale characterizing the propagation of the fire from one node to a neighboring node, namely, one temporal iteration. The next timescale is that associated with tree growth, and is given by $p_{\mathrm{g}}^{-1} = 10^3$ iterations here. Starting with an empty 100×100 lattice, this means that, on average, 10 new trees would grow at each temporal iteration. The first three snapshots in the last row exemplify quite well how much longer than the dynamical timescale this is; they must be scrutinized very carefully to notice the $\simeq$300 new trees that have appeared in the course of the 30 iterations spanned by these snapshots. The spontaneous activation probability—lightning strikes—usually determines the longest timescale. The expected time interval between two successive activations is of the order of $(p \times N^2 \times p_{\mathrm{f}})^{-1}$, where N is the linear size of the lattice and p the mean occupation probability in the statistically stationary state; this is defined as the number of live trees divided by N^2, the total number of lattice nodes.[2]

[2]Here, the occupation probability p is not an input parameter, as in percolation, but is a characteristic of the statistically stationary state attained by the simulation, but even then, only to a first approximation (more on this shortly).

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

Here, with $N = 100$, $p \sim 0.2$, and $p_f = 10^{-5}$, a lightning strike is expected every 50 iterations on average, but it must be kept in mind that, following a large fire such as in figure 6.2, p can fall much below its mean value calculated over the duration of the simulation.

The bottom-right panel in figure 6.2 shows the "cluster" of all trees burned in the 105-iteration-long fire covered by the other frames. Overall, this maps well, but not perfectly, to the tree density characterizing the top-left panel of figure 6.2, just prior to fire onset. Note how this cluster of burned trees contains "holes" in which clumps of trees have survived the fire, as the burning front became more convoluted. The shape of this cluster should remind you of the percolation clusters encountered in chapter 4; and yes, you hopefully guessed it, this cluster of burned trees is a fractal.

Figure 6.3 shows a segment of the time series of burning trees, in the same simulation. The large fire of figure 6.2 is the largest of the three fires visible on this time series, starting at iteration 3055. Fires clearly span a wide range in size, and their activity can show significant temporal variability in the course of a given fire. As one might have expected, large fires destroying large numbers of trees tend to burn longer and flare up more strongly, but the correlation between these fire measures is far from perfect; in figure 6.3, the third fire lasts only a few iterations more than the second, but destroys almost three times as many trees.

6.4 Model Behavior

The numerical choices made for the growth and activation probabilities p_g and p_f can lead to widely varying behaviors in the spatiotemporal evolution of the system. This is illustrated in figure 6.4 (and in figure 6.6 discussed later), which shows time series of the number of occupied nodes that are inactive, N_a (i.e., occupied by a tree; in green) and active, N_f (burning trees; in red), for four different combinations of p_g and p_f values.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

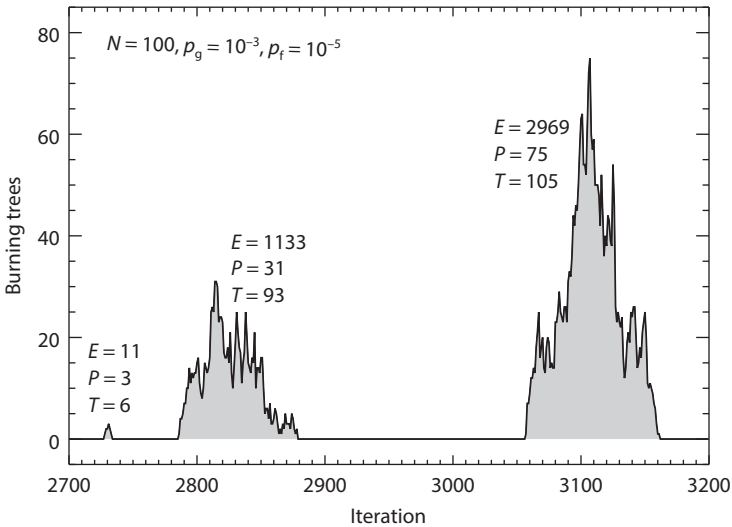

Figure 6.3. Time series of the number of burning trees in the simulation of figure 6.2, with the large fire starting at iteration 3055 being the one captured by that sequence of snapshots. Next to each fire are listed the total number of burned trees (E), the peak number of burned trees at any single iteration (P), and the fire duration (T).

If $p_f \sim p_g$ (top panel of figure 6.4, with $p_g = p_f = 10^{-4}$), then trees are struck by lightning at a frequency comparable to their growth rate. The total number of trees remains approximately constant, and numerous small fires are always burning here and there, without ever becoming large, because the density of trees is too small; with N_a hovering around 1500, only 15% of the 10^4 lattice nodes are occupied at any given time, meaning that few pairs of trees stand on neighboring nodes. If p_g is raised to 10^{-2} (bottom panel in figure 6.4), trees grow much faster and their density is roughly twice as much. Not only can fires now spread, but in fact trees are now growing so rapidly that once ignited, a fire never stops because new growth behind the

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

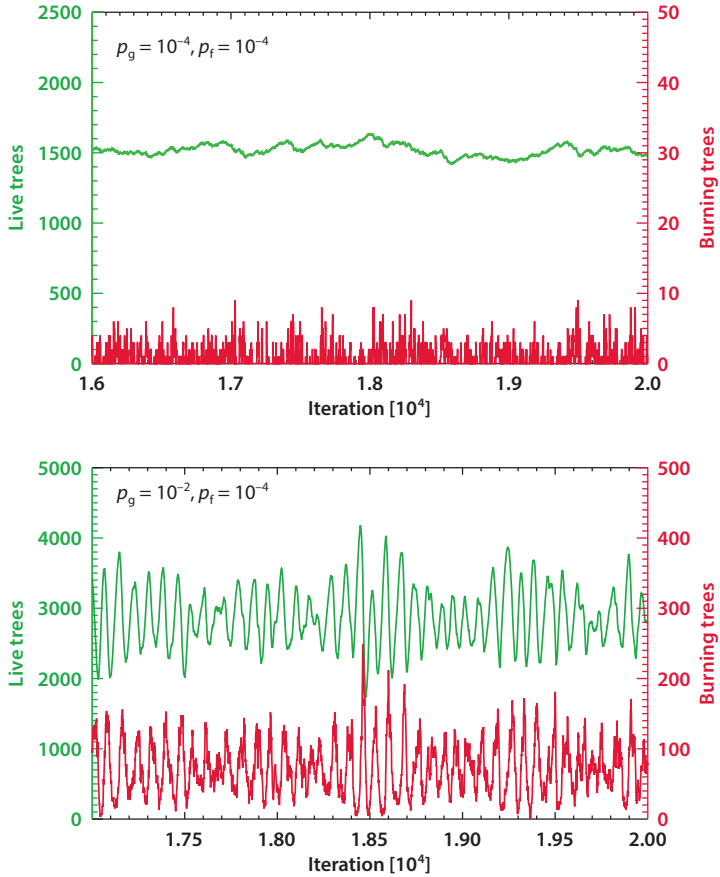

Figure 6.4. Time series for the number of active (red) and inactive (green) occupied nodes, for various combinations of p_g and p_f, in a regime where these growth and activation probabilities are relatively high. Both of these simulations are run on a 100×100 lattice, and the time series plotted are extracted far into the statistically stationary state.

burning front replenishes the forest at a rate comparable to the time it takes the fire to move across the lattice, here of the order of 100 iterations.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

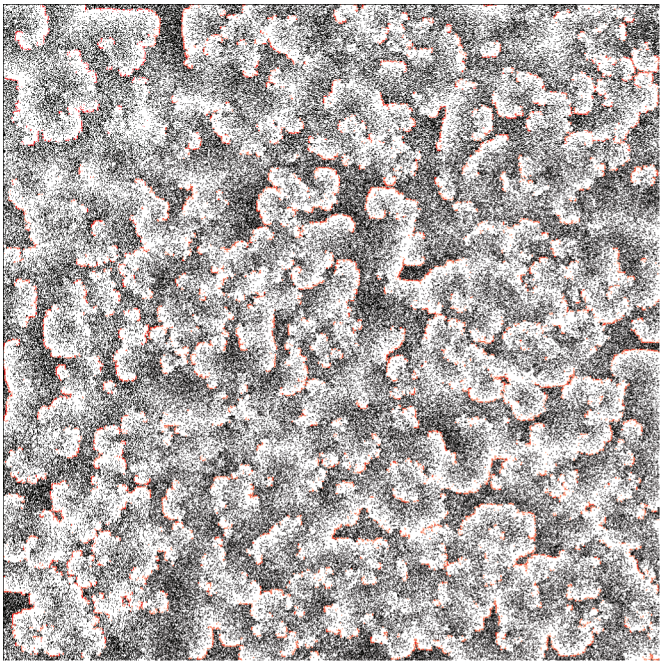

Figure 6.5. Snapshot of a 1024×1024 lattice, for a simulation with $p_g = 10^{-2}$ and $p_f = 10^{-5}$, but with lightning artificially turned off after the first 50 iterations. In this parameter regime, trees grow so fast that once ignited, fire persists throughout the simulation, with burning fronts expanding, fragmenting, shrinking, and interacting with one another. Note how the curved burning fronts often show a tendency to spiral inward at their extremities.

These parameter regimes are of course ecologically unrealistic, but represent classes of possible behavior for this model that are quite interesting in their own right. Figure 6.5 shows a snapshot of a 1024×1024 lattice in the rapid regrowth regime ($p_g = 10^{-2}$), where a few dozen random lightning strikes have taken place in the first 50 iterations, but lightning has been artificially "turned off" afterward. Moving burning fronts (in red) are ubiquitous across the lattice, growing, shrinking, fragmenting,

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

merging, and interacting with one another, and often develop into approximately circular arcs, with their tips curling back inward, in the manner of a spiral with a large opening angle.[3] The density of trees (in green) at any location on the lattice undergoes a recurrence cycle of slow growth at a rate set by p_g, up to a value approaching unity, then a sudden drop to zero as the burning front moves through, followed by slow growth anew. This general type of recurrence cycle will be encountered repeatedly in subsequent chapters.

Let's get back to the more ecologically realistic situation where trees grow slowly and fires are rare events. Figure 6.6 shows what happens if the growth and activation probabilities are lowered to the much smaller values $p_g = 10^{-3}$ and $p_f = 10^{-6}$. The lattice now has enough time to really fill up before lightning strikes again. But when the ignition finally happens, almost every tree has at least one nearest neighbor, so the fire sweeps almost the whole lattice clean. This leads to a quasiperiodic "load/unload" recurrence cycle whereby, at more or less regular intervals, the whole forest is destroyed, and regrowth must start from zero or nearly so. In the top panel of figure 6.6, when lightning strikes there are around 5000 occupied nodes, out of a possible grand total of $100 \times 100 = 10{,}000$. The corresponding occupation probability is therefore $\simeq 0.5$, which pretty much guarantees that every tree has a neighbor. The fact that a few hundred trees remain at the end of a large fire is in part a boundary effect; on a 100×100 lattice, 392 nodes are boundary nodes that have three fewer neighbors than interior nodes, and the 4 corner nodes have even fewer. These boundary nodes are thus harder to reach for an ongoing fire.

At low p_f, the only way to break the load/unload cycle so prominent in the top panel of figure 6.6 is if tree growth is

[3]In chapter 11, we will encounter a CA behaving similarly, when discussing excitable systems and reaction–diffusion chemical reactions.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

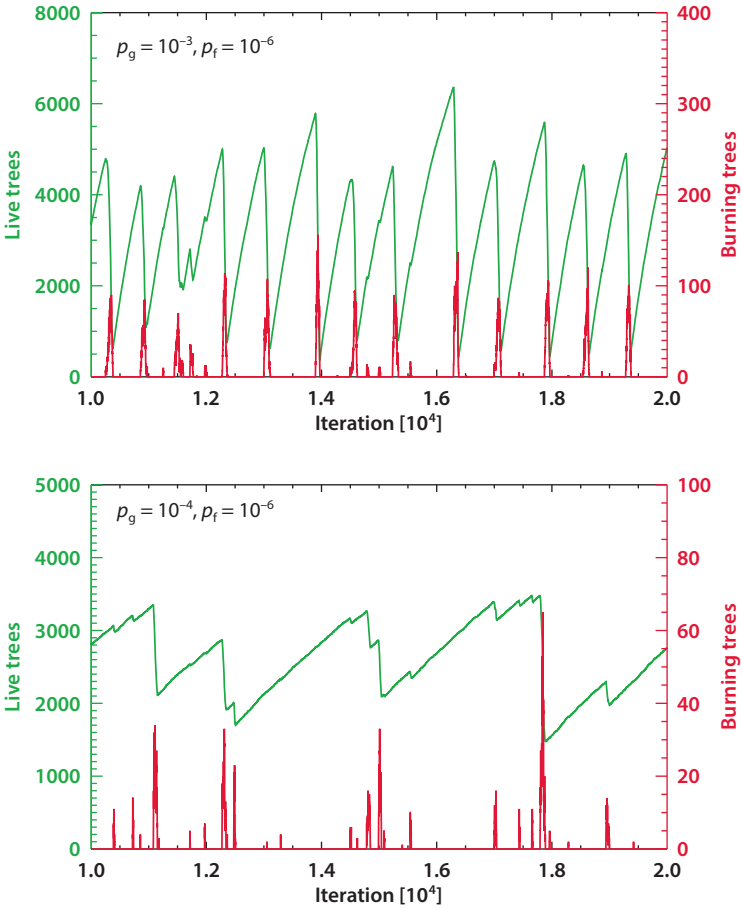

Figure 6.6. Identical in format to figure 6.4, but now for simulations operating in the regime where the activation probability p_f is very small. Compare the bottom plot to the inset in figure 5.4A.

sufficiently slow that the lattice does not have time to completely fill up between two successive lightning strikes. Keeping $p_f = 10^{-6}$ but lowering p_g to 10^{-4} yields the solution plotted in the bottom panel of figure 6.6. Fires, when they occur, can

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

still be quite large, but they are now triggered far less regularly and exhibit a wide range of sizes. Note also the fractal sawtooth pattern of the time series for occupied nodes, which shows an uncanny resemblance to the mass time series in the sandpile model of the preceding chapter (cf. the inset in figure 5.4A).

In cases like in figure 6.6, where one or more fires are not burning continuously somewhere on the lattice (as they do in figure 6.5), it is possible to characterize each individual fire as we did for avalanches in the sandpile model, through the variables E, P, and T, defined respectively as

1. E: the total number of trees burned in the fire;
2. P: the peak number of trees burned at any one iteration in the course of the fire;
3. T: the fire duration, measured in temporal iterations.

These quantities are correlated with one another, in that large fires tend to last longer, but we know already from figure 6.3 that a perfect correlation is not to be expected. Figure 6.7 shows the PDFs of fire sizes (E) for the two simulations of figure 6.6. At $p_g = 10^{-3}$ the distribution is approximately Gaussian, centered here around fire size 4800, but with a long, flat non-Gaussian tail extending to much smaller fires and a narrow, tall peak at very small fire sizes (off scale to the left in figure 6.7A). In this $p_f \ll 1$ regime, lowering the growth probability from $p_g = 10^{-3}$ to $p_g = 10^{-4}$ leads to a transition from a Gaussian distribution, with a relatively well-defined mean, to a power law of the form

$$f(E) = f_0 E^{-\alpha}, \quad \alpha > 0, \tag{6.1}$$

here with $\alpha = 1.07$. For such a power-law PDF it can be shown that the average fire size $\langle E \rangle$ is given by

$$\langle E \rangle = \frac{f_0}{2-\alpha}\left[E_{\max}^{2-\alpha} - E_{\min}^{2-\alpha}\right], \tag{6.2}$$

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

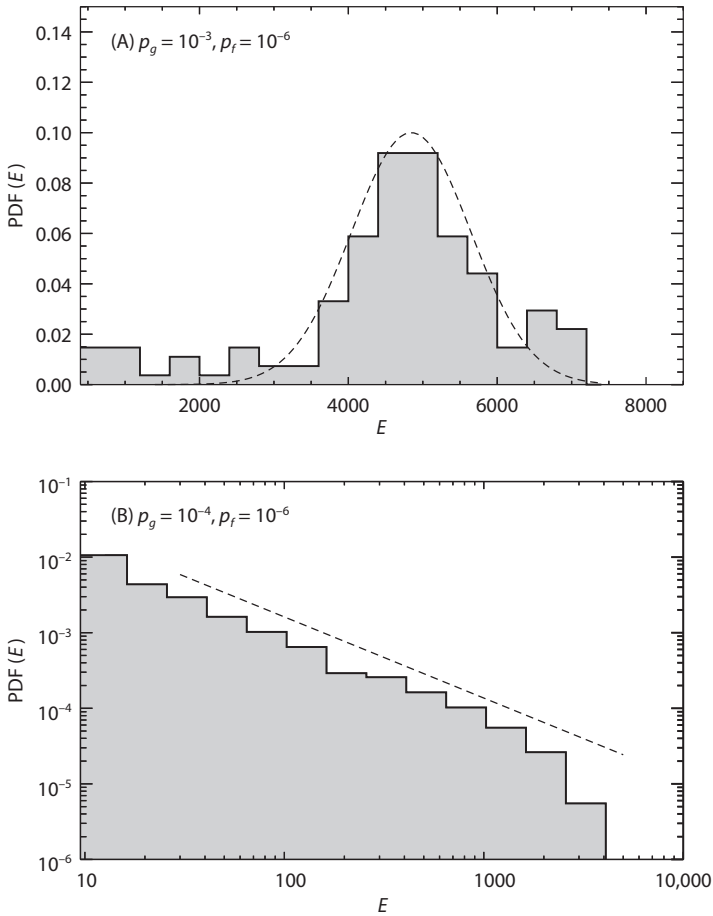

Figure 6.7. Probability distribution of fire size E for the two simulations of figure 6.6. The distribution in panel A is tolerably well fit by a Gaussian, except for its flat, low-amplitude tail extending to small fire sizes. The distribution in panel B is well described by a power law with index -1.07.

where $E_{\min}$ and $E_{\max}$ are the smallest and largest fires that can be produced by the simulation, here 1 and 10^4, respectively (see appendix B for the calculation of averages from a PDF). With $E_{\min} \ll E_{\max}$ (which is usually the case on large lattices) and

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

$\alpha < 2$, this is well approximated by

$$\langle E \rangle \simeq \frac{f_0 E_{\max}^{2-\alpha}}{2-\alpha}, \quad [\alpha < 2]. \tag{6.3}$$

This is because with $\alpha < 2$, the exponent $2 - \alpha$ in equation (6.2) is positive, so that the first term in the square brackets ends up much larger than the second. The opposite would be true if $\alpha > 2$. In the regime $p_g \ll 1$, $p_f \ll p_g$, the first case prevails, and therefore the largest fires dominate the evolution of the (eco)system.[4]

Whatever their shape, PDFs are defined such that $f(E)\,\mathrm{d}E$ measures the occurrence probability of a fire of size between E and $E + \mathrm{d}E$. In the ecologically realistic $p_g \ll 1$ regime, any one node contributes only one burned tree to a given fire; the situation was different in the sandpile model of the preceding chapter, where a node could topple repeatedly in the course of the same avalanche (see figure 5.5 if you're not convinced). Here, if a fire destroys E trees, it is because lightning hit somewhere within a cluster containing E connected trees. However, the probability that a cluster of size E is hit by randomly distributed lightning strikes is also proportional to the cluster size. Therefore, the PDF of cluster sizes must be distributed as $\propto E^{-(\alpha+1)}$ if the PDF of fire sizes is $\propto E^{-\alpha}$; with $\alpha = 1.07$ for the simulation of figure 6.7B, this implies that clusters of trees are distributed as a power law with index -2.07. Recall from chapter 4 that percolation clusters show a scale-invariant power-law size distribution only at the percolation threshold (see figure 4.7). Can we then conclude that the forest-fire lattice is at the percolation threshold?

It turns out to be significantly more complicated than that. Unlike in classical percolation, the tree density, equivalent to the occupation probability in percolation, is not constant across the lattice in the forest-fire model. This is illustrated in figure 6.8, showing a snapshot of the distribution of trees

[4]The same holds for the Earth's crust, with the largest earthquakes contributing the most to the relaxation of tectonic stresses; more on this in chapter 8.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

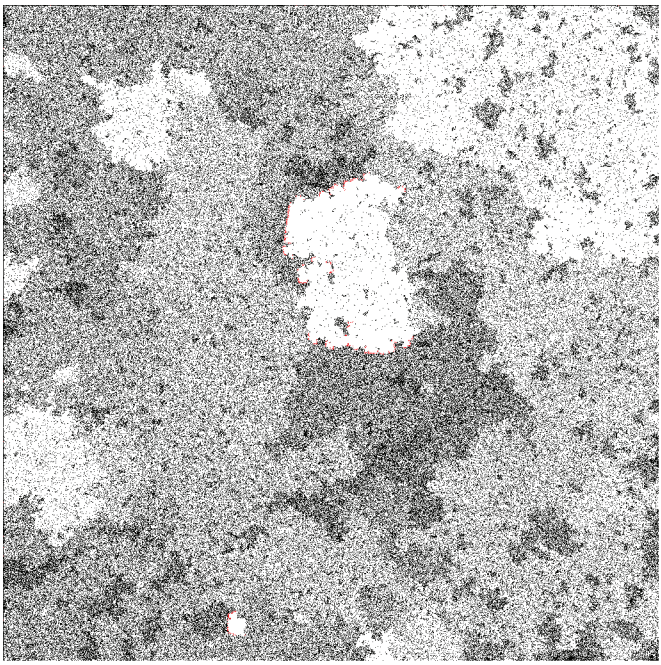

Figure 6.8. Snapshot of a forest-fire simulation on a 1024×1024 lattice, with parameters $p_g = 10^{-4}$ and $p_f = 10^{-7}$. Each small black dot is a tree, so that the resulting pointillistic gray shading provides a visual measure of tree density. Burning trees are plotted in red. Note how tree density, as measured visually by the level of gray shading, is approximately constant within contiguous domains, relatively well delineated but very irregularly shaped. The lighter areas are the scars of the more recent fires, and often contain dense clumps of surviving trees, corresponding to "holes" within the former clusters destroyed by fire. In this snapshot, two fires are burning, a large one near the lattice center and a smaller one near the bottom.

(black dots) in a $p_g = 10^{-4}$, $p_f = 10^{-7}$ simulation, now on a much larger 1024×1024 lattice. The mean density of trees is only approximately constant within irregularly shaped domains, with significant jumps occurring at the boundaries separating contiguous domains. These domains have been carved by prior

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

fires having swept through the lattice. Tree growth, as mediated by Rule 1, is random but statistically homogeneous in space, so that the mean density of a given (large enough) domain is proportional to the time elapsed since the end of the last major fire having swept through that domain. Each individual domain behaves effectively as a separate percolation lattice, with slowly increasing occupation probability. Immediately following a fire, the occupation probability is close to zero, but grows linearly with time, eventually reaching the percolation threshold ($p_c = 0.4072$ for a Cartesian lattice with 8-neighbor connectivity). Recall that the likelihood of a single cluster taking over the lattice increases very rapidly once moving beyond this threshold (see figure 4.6), so that lightning, when and wherever it hits, is likely to wipe out the whole domain in a single fire. The shape and size of domains evolve slowly in the course of the simulation, because part of a domain may be destroyed by fire before reaching the percolation threshold (lightning hitting "early"), or by fusion with neighboring domains if both have exceeded significantly the percolation threshold prior to one igniting (lightning hitting "late"). Clearly, the PDF of fire sizes is determined by the past history of fires, going back at least a few p_g^{-1} iterations. Since typically $p_g \ll 1$, the system is said to exhibit long temporal correlations.

6.5 Back to Criticality

Running forest-fire model simulations for various combinations of growth and activation probabilities p_g and p_f, one soon realizes that in the portion of parameter space satisfying the double limit

$$p_f \ll p_g, \quad p_g \ll 1, \tag{6.4}$$

the PDF of fire sizes (and durations) always assumes a power-law shape. Moreover, in that regime the power-law index is always the same, and, for large enough lattices, is independent of lattice size. In other words, the corresponding values of α are universal, and

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

involve no fine-tuning of control parameters. In this regime, the forest-fire model exhibits SOC.

In terms of the conditions for SOC behavior identified at the end of chapter 5, slow forcing is tree growth; the threshold instability (with respect to lightning strike) is the presence of a tree on the node hit by lightning; redistribution is the propagation of fire to neighboring trees. The system is open, because new trees are continuously added to the lattice, and dissipative, because a mechanism (fire) removes trees from the lattice.

But why should it matter whether wildfires represent an instance of SOC? It turns out to matter a lot when you decide to actively manage wildfires.

6.6 The Pros and Cons of Wildfire Management

As I write these lines, life is slowly returning to the 1,500,000+ acres of land (over 6000 square kilometers) charred by the spring 2016 Fort McMurray wildfire in Northern Alberta. It is currently lining up to rank as the costliest natural disaster in Canadian history. Amazingly enough, "only" two people died, in a car collision during the town's evacuation. Sometimes the toll gets worse. I lived in Colorado back in 1994 and vividly recall the Storm King Mountain wildfire near Glenwood Springs, which on July 6 claimed the lives of 14 firefighters who could not escape a rapidly moving fire front. And if this was not bad enough, a century ago an estimated 223 Northern Ontario residents suffered the same fate when half a dozen small communities were swept by the July 29, 1916 Matheson wildfire. The dangers of wildfires, and wildfire fighting, are not to be taken lightly. This is serious business.

In Canada, as in the United States, until recently and to some extent still now, wildfire management consisted in putting out potentially dangerous wildfires as quickly as possible, when the fire is still small. It sure seems to make a lot of sense. This type of fire management practice is easy to incorporate in the

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

simulation code of figure 6.1. For example, introduce a time-dependent *extinction probability* (p_e) which decreases with the current number of burning trees (n_b) as

$$p_e(t) = \begin{cases} 0.2/n_b(t) & \text{if } n_b \leq 10, \\ 0 & \text{otherwise.} \end{cases} \tag{6.5}$$

Now when a fire is triggered and begins to grow, at every subsequent temporal iteration a probability test forces simultaneous extinction of all burning nodes with probability p_e. As the fire grows beyond 10 simultaneously burning nodes, this probability will become zero, reflecting the fact that real wildfires become very hard to extinguish once they really get going.

Considering that even large fires start off small, this procedure will clearly reduce the number of fires burning on the lattice over a set time span. Since the PDF of fire sizes has a power-law shape, however, most extinguished fires would have remained small anyway. Extinguishing them thus leaves more fuel for subsequent fires; whenever one manages to grow to a size where the probability of being extinguished goes to zero, as per equation (6.5), the forest is more densely packed with trees than it would have been had the earlier small fires not been extinguished. As a consequence, the total number of fires decreases, but the size of the largest fires may well increase! Not at all the desired outcome of good wildfire management. The Grand Challenge for this chapter leads you through a quantitative investigation of this phenomenon.

6.7 Exercises and Further Computational Explorations

1. The time series at the bottom of figure 6.4 shows very clear periodicity; can you determine what sets the period here?

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

2. Run two forest-fire simulations using the parameter values in figure 6.6. Make sure to run your simulations long enough to generate a few hundred fires at least. Calculate the fire measures E, P, and T, as in figure 6.3, and examine how these correlate against one another for your ensemble of fires. In both cases examine also whether fire size E correlates with the time elapsed since the end of the previous fire, or with the size of the previous fire.
3. The aim here is to have you test some modifications to the forest-fire model, and examine their impact. Work with a 100×100 lattice, and try at least one of the following (and the more the better!):

    a. Modify the Python source code of figure 6.1 so that it operates in "stop-and-go" rather than "running" mode, i.e., no tree is allowed to grow as long as a fire is burning anywhere on the lattice. In which parts of parameter space does this alter the global behavior of the model?
    b. Modify the Python source code of figure 6.1 so that fire propagates only to the 4 nearest neighbors top/down/right/left. Does this alter the global behavior of the model?
    c. Modify the Python source code of figure 6.1 so that the growth probability increases linearly with the number n of occupied neighboring nodes, for example $p_g \to p_g(1 + n)$. Does this alter the global behavior of the model?
    d. Modify the Python source code of figure 6.1 to introduce periodic boundary conditions horizontally and vertically (see appendix D for more detail on implementing such boundary conditions on a

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

lattice). Set $p_f = 10^{-5}$ and explore the types of patterns generated at $p_g = 10^{-3}$ and 10^{-2}.

4. The forest-fire model is ideally suited to investigating an interesting variation on percolation, sometimes known as *dynamical percolation*. The idea is to replace the initial condition in the forest-fire model of figure 6.1 by a classical percolation lattice with occupation probability p (see the small Python code at the beginning of section 4.2). Now turn off tree growth and lightning, but as an initial condition, set on fire all nodes along the left edge of the lattice, and run the model until the fire extinguishes. Repeat the process for 10 distinct random realizations of your percolation lattice, and keep track of the fraction of runs for which the fire reaches the right edge prior to extinction. Repeat for varying p, and construct a plot showing the fraction of "successful" realizations versus p. How does this plot compare to the bottom panel in figure 4.6? How would you estimate the percolation threshold p_c from such ensemble averaging?
5. The numerical implementation of the forest-fire model listed in figure 6.1 is extremely inefficient in many respects. For example, just consider the fact that every empty node on the lattice is subjected to the tree growth probability test at every temporal iteration; for an $N \times N$ lattice, since trees grow only on empty nodes (Rule 1), it would be much faster to "grow" a tree at $p_g \times N_e$ randomly selected empty nodes, where N_e is the number of empty nodes at the current iteration. Modify the Python source code of figure 6.1 to operate in this manner. And, if you feel up to some more serious coding, see exercise 5 in chapter 5 for more ideas.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

6. And finally for the Grand Challenge: wildfire mitigation and management! The idea is to implement the strategy outlined in section 6.6 into the basic code of figure 6.1. Work off a 128×128 lattice in the SOC regime of figure 6.6: $p_g = 10^{-4}$ and $p_f = 10^{-6}$. Examine how the PDF of fire sizes varies as you increase the probability of extinction, i.e., replace the numerical factor 0.2 in equation (6.5) by the values 0.1, 0.2, 0.3, and 0.5. Run the simulations for the same number of temporal iterations in all cases. Is the PDF getting steeper or flatter as the probability of extinction increases? How about the size of the largest fires? How would you go about designing an "optimal" wildfire management strategy in the context of this model? Note that in the context of this Grand Challenge, you will be trying to obtain accurate determinations of the power-law index of the PDFs at each extinction probability; you may consider calculating this index following the maximum likelihood approach described in appendix B.6. Make sure to exclude the initial transient phase from your analyses, and to push the simulations far enough in time to have many hundreds of fires to build your PDFs from, even in the simulation with the fewest fires.

6.8 Further Reading

The forest-fire model introduced in this chapter is due to

Drossel, B., and Schwabl, F., "Self-organized critical forest-fire model," *Phys. Rev. Lett.*, **69**(11), 1629–1632 (1992).

A comprehensive review of its properties can be found in

Hergarten, S., "Wildfires and the forest-fire model," in *Self-organized Criticality Systems*, ed. M.J. Aschwanden, Berlin: Open Academic Press, 357–378 (2013).

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms

I know of no good textbook dedicated to the mathematical modeling of wildfires, but the topic is sometimes covered in textbooks on mathematical modeling in general. I did find the Wikipedia page on wildfire modeling well balanced and quite informative, and it also includes many good references to the technical literature (consulted November 2014):

http://en.wikipedia.org/wiki/Wildfire_modeling.

On the comparison of real wildfire data with a suitably tailored SOC-type model akin to that considered in this chapter, including fire management strategies, see

Yoder, M.R., Turcotte, D.L., and Rindle, J.B., "Forest-fire model with natural fire resistance," *Phys. Rev. E*, **83**, 046118 (2011).

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:08 UTC
All use subject to https://about.jstor.org/terms